Predicting Vehicle Prices Using Regression Models

Objectives

The primary goal of this project is to develop a robust regression model to predict used car prices for a reseller based on various listed features and specifications. In addition to predicting prices, the project focuses on identifying feature importance and mitigating overfitting through the application of regularization techniques.

There can be several business objectives for this, such as:

- **Price Prediction**: Model car prices based on features like mileage, fuel type, and performance.
- **Market Analysis**: Explore trends and preferences in the used car market, by type, region, or other metrics.
- **Feature Importance**: Identify the most important factors influencing car prices (e.g., fuel type, mileage, age).

Tasks Overview

The data pipeline for this task involves the following steps:

1. **Dataset Overview**
2. **Data Preprocessing**
3. **Data Visualizations & Exploration**
4. **Model Building**
5. **Regularization**

1 Data Loading

1.1.1

Load the dataset

Load the data
df = pd.read_csv('/content/Car_Price_data.csv')
print(f"Dataset Shape: {df.shape}")
display(df.head())

2 Analysis and Feature Engineering [35 marks]

2.1 Preliminary Analysis and Frequency Distributions [13 marks]

2.1.1 *[1 marks]*

Check and fix missing values.

```
# Find the proportion of missing values in each column and handle if found

missing_percentages = df.isnull().mean() * 100
print("Percentage of missing values per column:")
print(missing_percentages[missing_percentages > 0])

# Impute missing values:
# - For numerical columns, we can fill with the median
# - For categorical columns, we can fill with the mode
for col in df.columns:
    if df[col].isnull().any():
        if df[col].dtype in ['int64', 'float64']:
            df[col] = df[col].fillna(df[col].median())
        else:
            df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing values after imputation:", df.isnull().sum().sum())
```

```
Percentage of missing values per column:
Series([], dtype: float64)

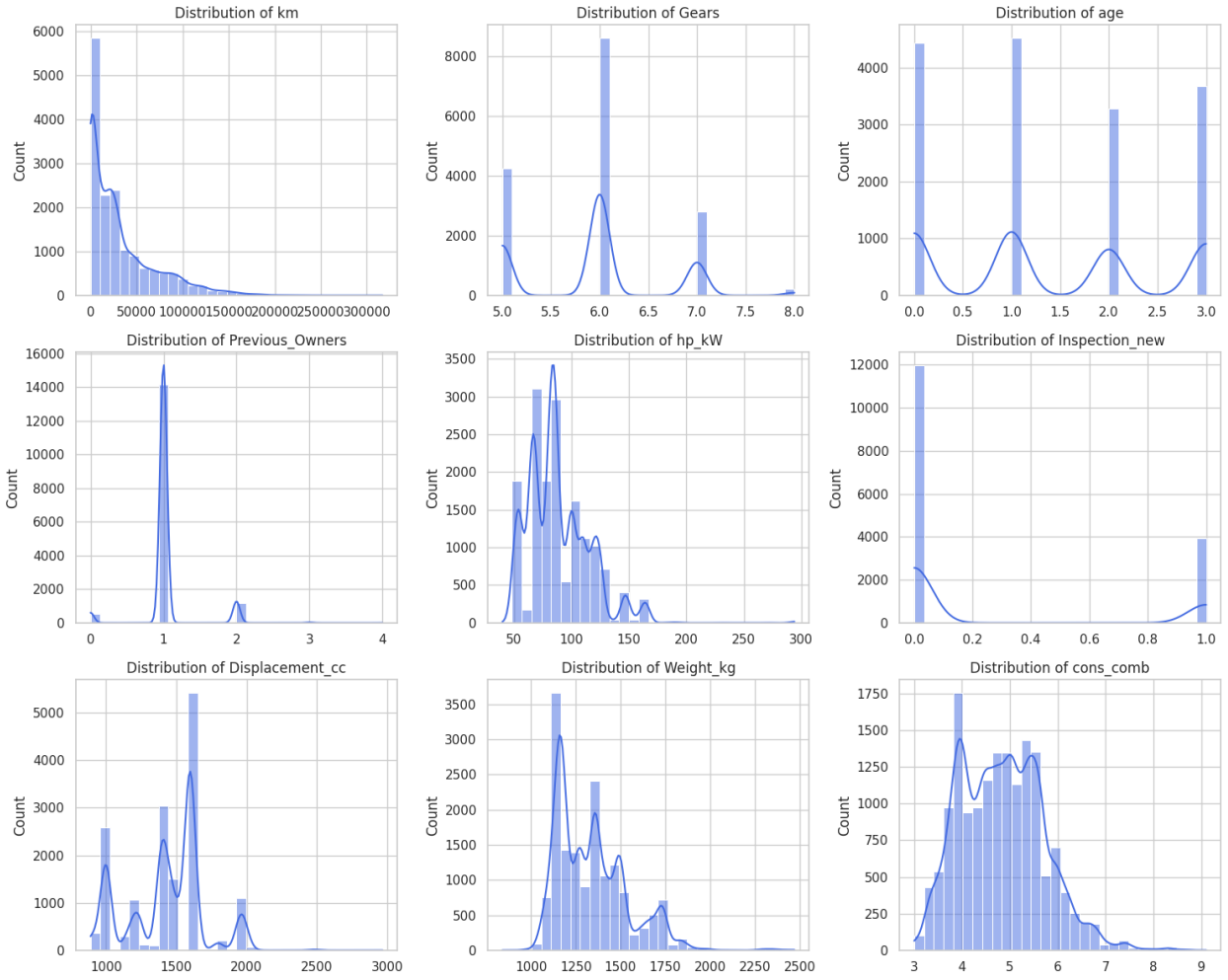
Missing values after imputation: 0
```

From the features, identify the target feature and numerical and categorical predictors. Select the numerical and categorical features carefully as they will be used in analysis.

2.1.2 *[3 marks]*

Identify numerical predictors and plot their frequency distributions.

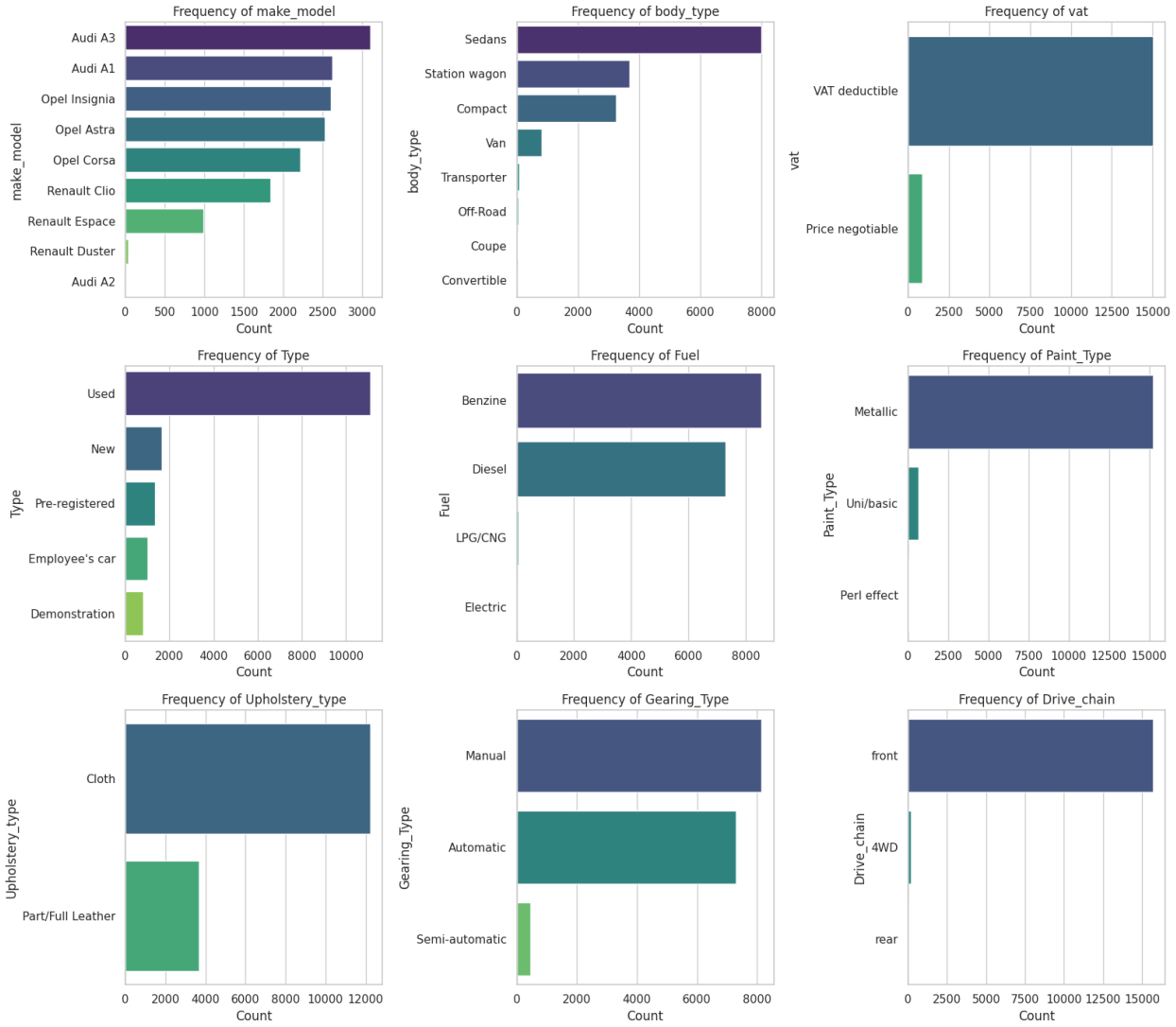
Numerical Predictors: ['km', 'Gears', 'age', 'Previous_Owners', 'hp_kW', 'Inspection_new', 'Displacement_cc', 'Weight_kg', 'cons_comb']



2.1.3 [3 marks]

Identify categorical predictors and plot their frequency distributions.

Categorical Predictors: ['make_model', 'body_type', 'vat', 'Type', 'Fuel', 'Paint_Type', 'Upholstery_type', 'Gearing_Type', 'Drive_chain']



Note: Look carefully at the values stored in columns ["Comfort_Convenience", "Entertainment_Media", "Extras", "Safety_Security"].

Should they be considered categorical? Should they be dropped or handled any other way?

Answer: *No. They have multi-label / set-valued features, not single-category variables. We will treat the list columns by splitting the comma-separated strings and performing multi-label indicator encoding*

2.1.4 [3 marks]

Fix columns with low frequency values and class imbalances.

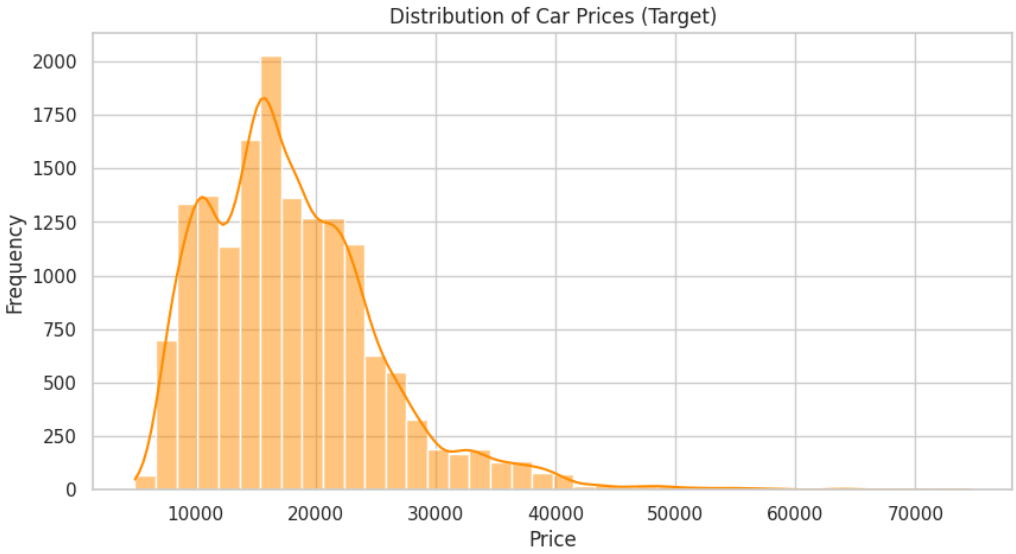
```
Original 'Type' counts:
 Type
Used              11095
New                1649
Pre-registered     1364
Employee's car     1011
Demonstration       796
Name: count, dtype: int64
Original 'Drive_chain' counts:
 Drive_chain
front    15707
4WD        204
rear         4
Name: count, dtype: int64

Updated 'Type' counts:
 Type
Used            12106
New_and_Demo     3809
Name: count, dtype: int64

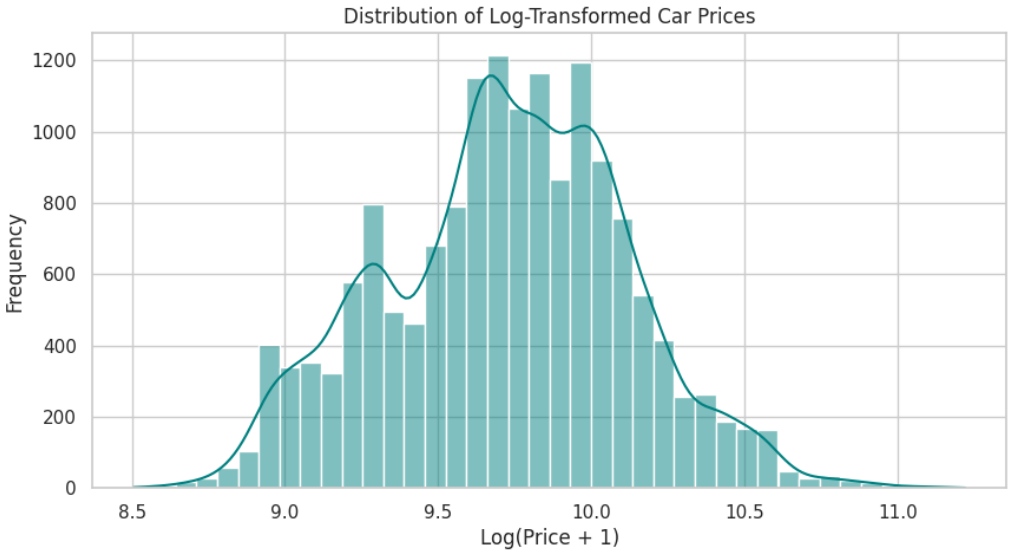
Updated 'Drive_chain' counts:
 Drive_chain
front_or_rear    15711
4WD                204
Name: count, dtype: int64
```

2.1.5 *[3 marks]*

Identify target variable and plot the frequency distributions. Apply necessary transformations.



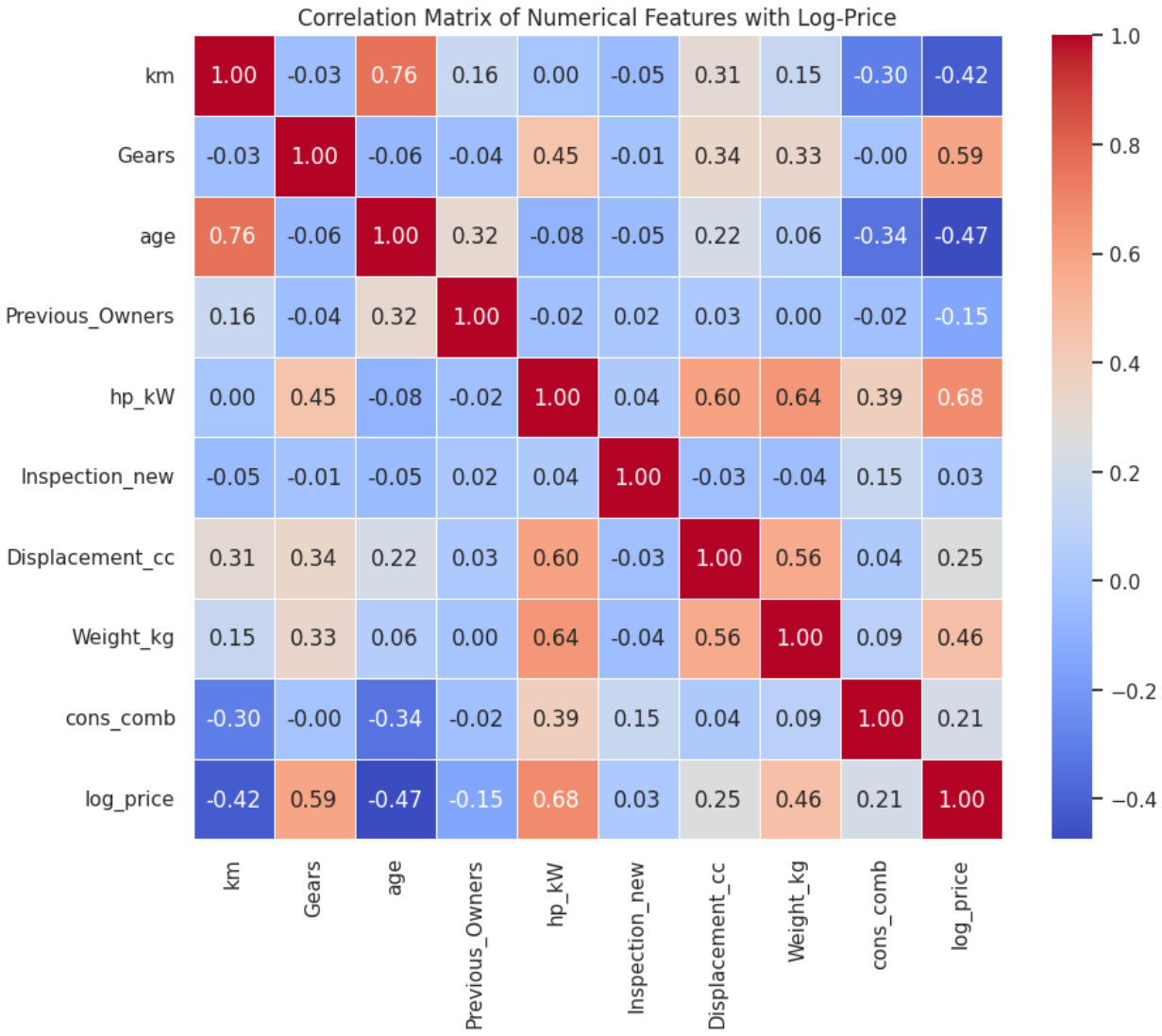
The target variable seems to be skewed. Perform suitable transformation on the target.



2.2 Correlation analysis [6 marks]

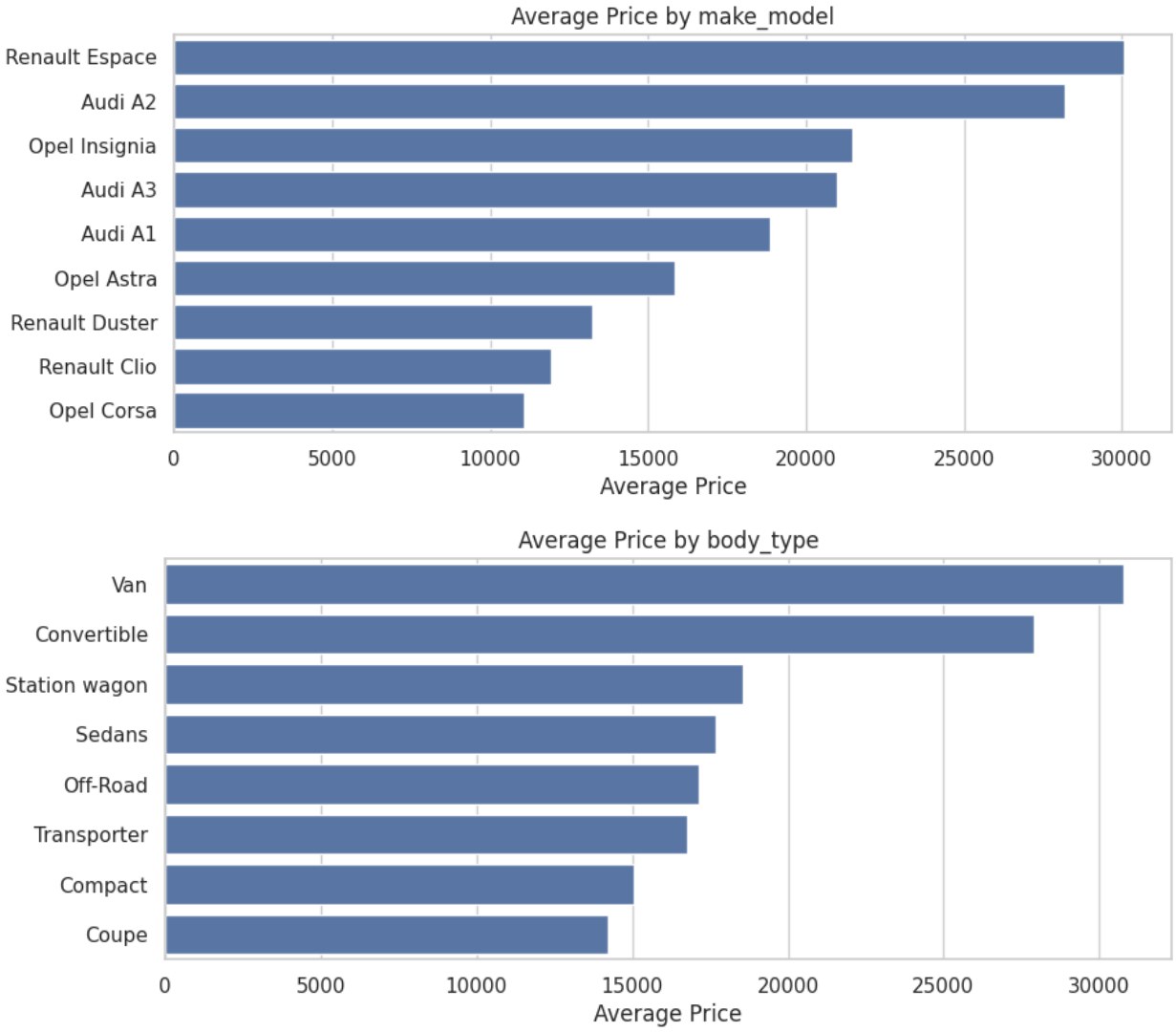
2.2.1 *[3 marks]*

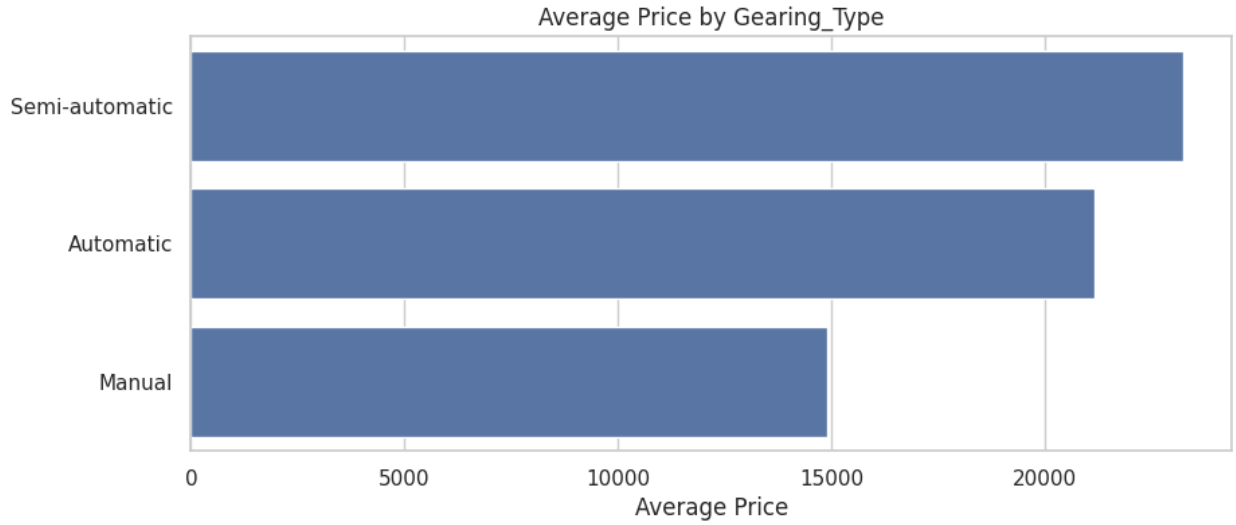
Plot the correlation map between features and target variable.



2.2.2 *[3 marks]*

Analyse correlation between categorical features and target variable.





2.3 Outlier analysis [5 marks]

2.3.1 *[2 marks]*

Identify potential outliers in the data.

```
# Outliers present in each column
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"{col}: {len(outliers)} outliers detected (lower limit: {lower_bound:.2f}, upper limit: {upper_bound:.2f})")
```

```
km: 689 outliers detected (lower limit: -65548.75, upper limit: 114369.25)
Gears: 225 outliers detected (lower limit: 3.50, upper limit: 7.50)
age: 0 outliers detected (lower limit: -3.00, upper limit: 5.00)
Previous_Owners: 1757 outliers detected (lower limit: 1.00, upper limit: 1.00)
hp_kW: 361 outliers detected (lower limit: 10.50, upper limit: 158.50)
Inspection_new: 3932 outliers detected (lower limit: 0.00, upper limit: 0.00)
Displacement_cc: 21 outliers detected (lower limit: 675.50, upper limit: 2151.50)
Weight_kg: 87 outliers detected (lower limit: 704.50, upper limit: 1932.50)
cons_comb: 125 outliers detected (lower limit: 2.15, upper limit: 7.35)
```

2.3.2 [3 marks]

Handle the outliers suitably.

```
# Handle outliers
Q1 = df[num_cols].quantile(0.25)
Q3 = df[num_cols].quantile(0.75)
IQR = Q3 - Q1
lower_bounds = Q1 - 1.5 * IQR
upper_bounds = Q3 + 1.5 * IQR

df[num_cols] = df[num_cols].clip(lower=lower_bounds, upper=upper_bounds, axis=1)
print("Outliers handled successfully via optimized vectorized clipping!")
```

```
Outliers handled successfully via optimized vectorized clipping!
```

2.4 Feature Engineering [11 marks]

2.4.1

Fix any redundant columns and create new ones if needed.

```
# FIX/CREATE COLUMNS AS NEEDED
REDUNDANT_COLS = ['PRICE']
FOR COL IN REDUNDANT_COLS:
    IF COL IN DF.COLUMNS:
        DF.DROP(COLUMNS=[COL], INPLACE=TRUE)
PRINT("REDUNDANT COLUMNS DROPPED. CURRENT DATASET COLUMNS:", DF.COLUMNS.TOLIST())
```

```
REDUNDANT COLUMNS DROPPED. CURRENT DATASET COLUMNS: ['MAKE_MODEL', 'BODY_TYPE', 'VAT', 'KM',
'TYPE', 'FUEL', 'GEARS', 'COMFORT_CONVENIENCE', 'ENTERTAINMENT_MEDIA', 'EXTRAS',
'SAFETY_SECURITY', 'AGE', 'PREVIOUS_OWNERS', 'HP_KW', 'INSPECTION_NEW', 'PAINT_TYPE',
'UPHOLSTERY_TYPE', 'GEARING_TYPE', 'DISPLACEMENT_CC', 'WEIGHT_KG', 'DRIVE_CHAIN', 'CONS_COMB',
'LOG_PRICE']
```

2.4.2 [4 marks]

Analysis and feature engineering on ['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security'].

These columns contain lists of features present. Decide on how to include these features in the predictors.

```
# Check unique values in each feature spec column
spec_cols = ['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security']

for col in spec_cols:
    # Fill missing list specifications with empty string if any
    df[col] = df[col].fillna('')
    # Get all unique elements
    unique_elements = set()
    for row in df[col]:
        unique_elements.update([item.strip() for item in row.split(',') if item.strip()])

    # Create a binary column for each feature
    for element in list(unique_elements)[:15]: # Li to top 15 most frequent to avoid high dimensionality
        col_name = f"{col}_{element.replace(' ', '_')}"
        df[col_name] = df[col].apply(lambda x: 1 if element in x else 0)


print("Spec columns processed. New DataFrame shape:", df.shape)
```

```
Spec columns processed. New DataFrame shape: (15915, 74)
```

Out of these features, we will check the ones which are present in most of the cars or are absent from most of the cars. These kinds of features can be removed as they just increase the dimensionality without explaining the variance.

```
# Drop features from df
df.drop(columns=spec_cols, inplace=True)
print("Ready for encoding, splitting, and scaling.")
```

```
Ready for encoding, splitting, and scaling.
```

2.4.3 [3 marks]

Perform feature encoding.

```
# Encode features
cat_to_encode = [col for col in cat_cols if col in df.columns]

# Apply One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=cat_to_encode, drop_first=True)
print(f"Encoded DataFrame Shape: {df_encoded.shape}")
```

```
Encoded DataFrame Shape: (15915, 91)
```

2.4.4 [2 marks]

Split the data into training and testing sets.

Split data into 80%-20%

```
# Split data
X = df_encoded.drop(columns=['log_price'])
y = df_encoded['log_price']

# Perform train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"X_train shape: {X_train.shape}, X_test shape: {X_test.shape}")
```

```
X_train shape: (12732, 90), X_test shape: (3183, 90)
```

2.4.5 *[2 marks]*

Scale the features.

```
# Scale features
scaler = StandardScaler()

# Scale only numerical predictors
num_cols_to_scale = [col for col in num_cols if col in X_train.columns]

X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[num_cols_to_scale] = scaler.fit_transform(X_train[num_cols_to_scale])
X_test_scaled[num_cols_to_scale] = scaler.transform(X_test[num_cols_to_scale])

print("Features scaled successfully!")
```

```
Features scaled successfully!
```

3 Linear Regression Models [35 marks]

3.1 Baseline Linear Regression Model [10 marks]

3.1.1 *[5 marks]*

Build and fit a basic linear regression model. Perform evaluation using suitable metrics.

```
# Initialise and train model
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)
print("Baseline Linear Regression Model trained successfully!")
```

```
Baseline Linear Regression Model trained successfully!
```

```
# Evaluate the model's performance
y_pred_train = lr_model.predict(X_train_scaled)
y_pred_test = lr_model.predict(X_test_scaled)

# Evaluate the model on transformed log scale
mae_train = mean_absolute_error(y_train, y_pred_tra
mae_test = mean_absolute_error(y_test, y_pred_test)
rmse_train = np.sqrt(mean_squared_error(y_train, y_pred_train))
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
r2_train = r2_score(y_train, y_pred_train)
r2_test = r2_score(y_test, y_pred_test)

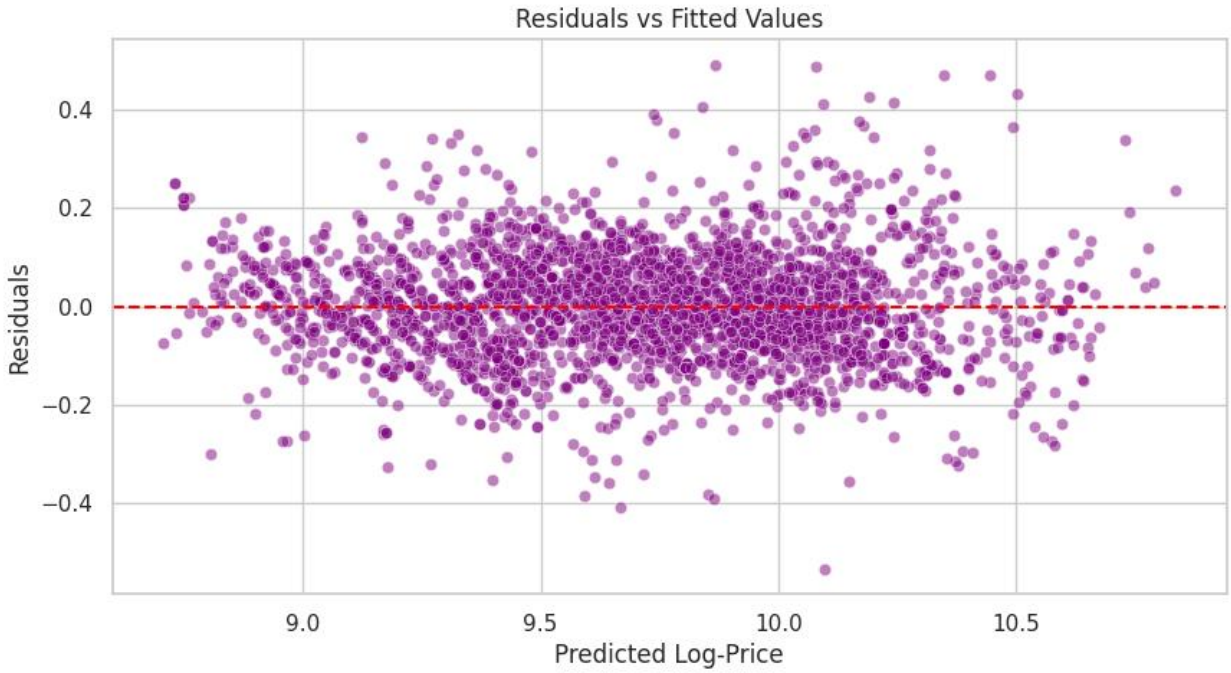
print(f"--- Baseline Linear Regression Performance ---")
print(f"Train MAE: {mae_train:.4f} | Test MAE: {mae_test:.4f}")
print(f"Train RMSE: {rmse_train:.4f} | Test RMSE: {rmse_test:.4f}")
print(f"Train R2: {r2_train:.4f} | Test R2: {r2_test:.4f}")
```

```
--- Baseline Linear Regression Performance ---
Train MAE: 0.0800 | Test MAE: 0.0793
Train RMSE: 0.1088 | Test RMSE: 0.1050
Train R2: 0.9252 | Test R2: 0.9311
```

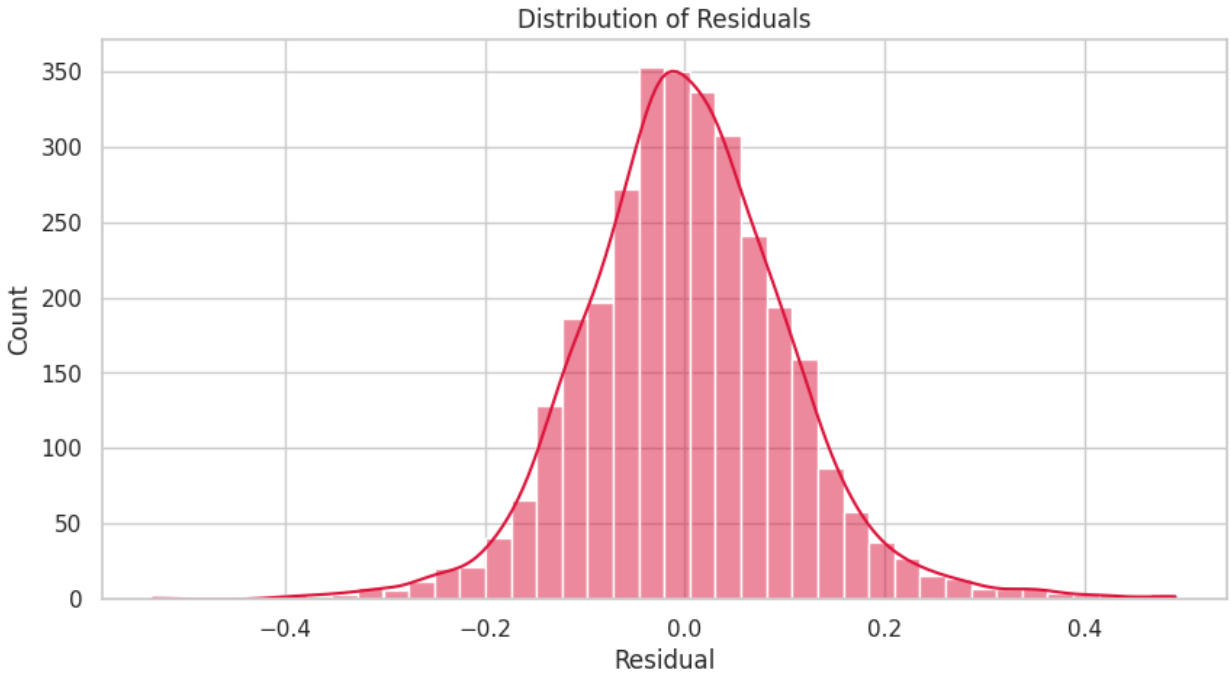
3.1.2 *[5 marks]*

Analyze residuals and check other assumptions of linear regression.

Check for linearity by analyzing residuals vs predicted values



Check normality in residual distribution



Check multicollinearity using Variance Inflation Factor (VIF) and handle features with high VIF.

```
Dropped constant features: ['make_model_Audi A2', 'Previous_Owners', 'Inspection_new']
```

	feature	VIF
3	hp_kW	2.993524
0	km	2.930200
2	age	2.802516
5	Weight_kg	2.046849
4	Displacement_cc	1.978208
6	cons_comb	1.479832
1	Gears	1.346877

3.2 Ridge Regression Implementation [10 marks]

3.2.1 *[2 marks]*

Define a list of random alpha values

```
# List of alphas to tune for Ridge regularisation
alphas_ridge = [0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 200.0, 500.0, 1000.0]
```

3.2.2 [4 marks]

Apply Ridge Regularization and find the best value of alpha from the list

```
# Applying Ridge regression

from sklearn.model_selection import cross_val_score

ridge_train_scores = []
ridge_test_scores = []

for alpha in alphas_ridge:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train_scaled, y_train)

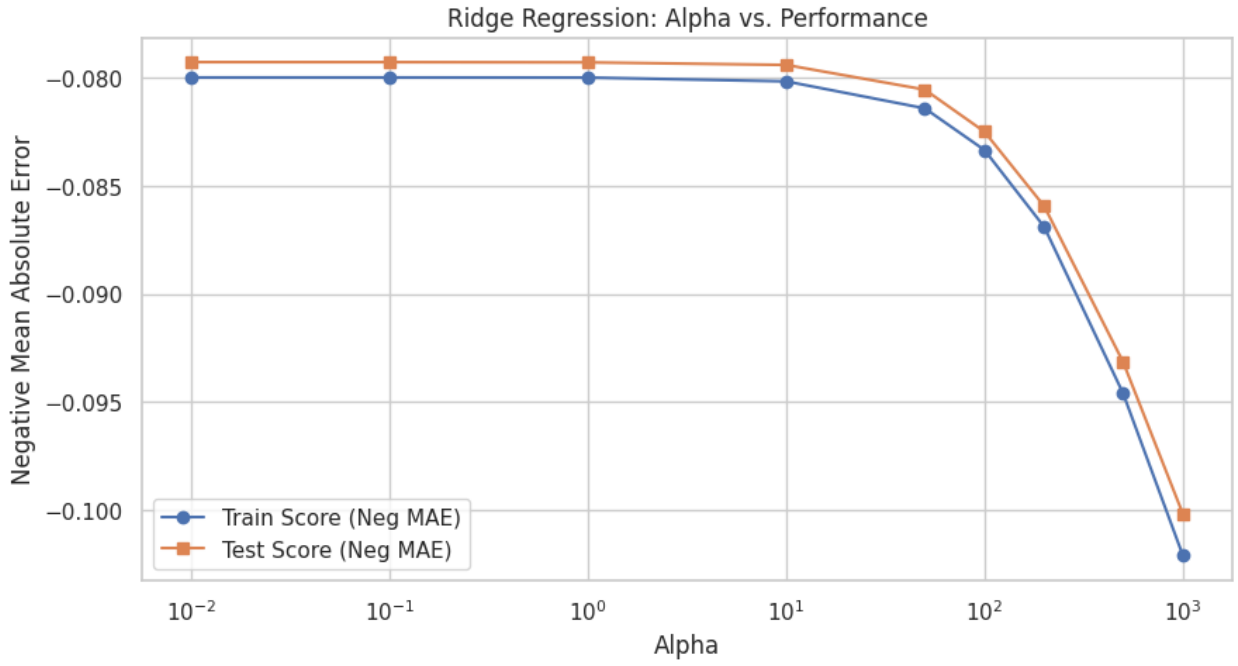
    # Evaluate using negative MAE
    train_pred = ridge.predict(X_train_scaled)
    test_pred = ridge.predict(X_test_scaled)

    ridge_train_scores.append(-mean_absolute_error(y_train, train_pred))
    ridge_test_scores.append(-mean_absolute_error(y_test, test_pred))

print("Ridge regression completed for initial alpha list.")
```

```
Ridge regression completed for initial alpha list.
```

Plot train and test scores against alpha



Find the best alpha value.

Best alpha value: 0.01

Best score (negative MAE): -0.0793

We will get some of the best value of alpha above. This, however, is not the most accurate value but the best value from the given list. Now we have a rough estimate of the range that best alpha falls in. Let us do another iteration over the values in a smaller range.

3.2.3 *[4 marks]*

Fine tune by taking a closer range of alpha based on the previous result.

```
# Take a smaller range of alpha to test
alphas_ridge_fine = np.linspace(0.1, 50.0, 50)

# Applying Ridge regression
ridge_train_scores_fine = []
ridge_test_scores_fine = []

for alpha in alphas_ridge_fine:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train_scaled, y_train)

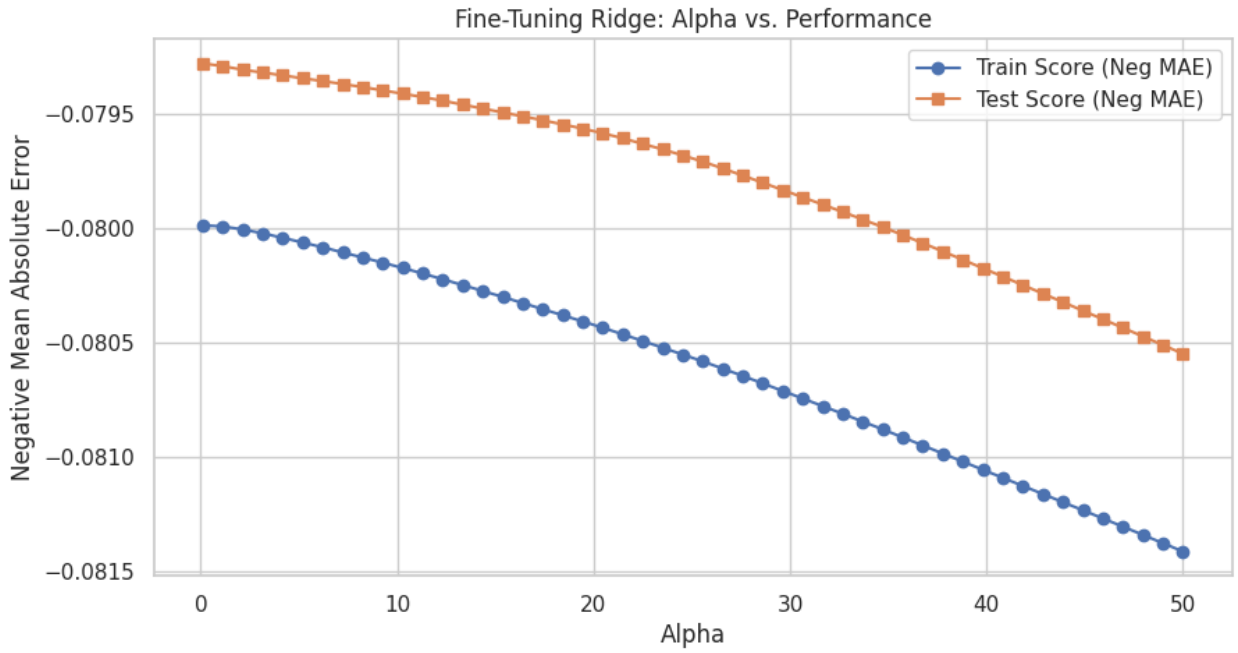
    train_pred = ridge.predict(X_train_scaled)
    test_pred = ridge.predict(X_test_scaled)

    ridge_train_scores_fine.append(-mean_absolute_error(y_train, train_pred))
    ridge_test_scores_fine.append(-mean_absolute_error(y_test, test_pred))
```

Plot the error-alpha graph again and find the actual optimal value for alpha.

Best alpha value: 0.1

Best score (negative MAE): -0.0793



Ridge Regression Performance (alpha=0.10)

MAE: 0.0793

RMSE: 0.1050

R2 Score: 0.9311

3.3 Lasso Regression Implementation [10 marks]

***3.3.1** [2 marks]*

Define a list of random alpha values

```
# List of alphas to tune for Lasso regularisation
alphas_lasso = [1e-5, 1e-4, 1e-3, 0.01, 0.1, 1.0, 10.0]
```

3.3.2 [4 marks]

Apply Ridge Regularisation and find the best value of alpha from the list

```
# Initialise Lasso regression model
lasso_train_scores = []
lasso_test_scores = []

for alpha in alphas_lasso:
    lasso = Lasso(alpha=alpha, max_iter=5000)
    lasso.fit(X_train_scaled, y_train)

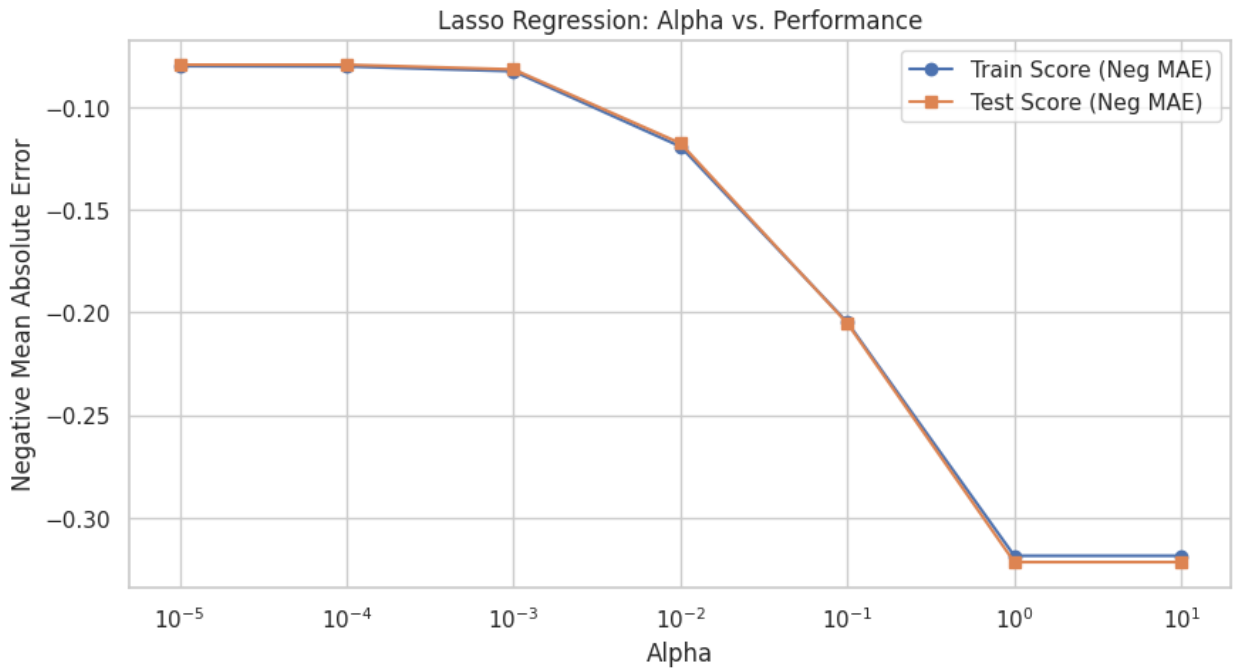
    train_pred = lasso.predict(X_train_scaled)
    test_pred = lasso.predict(X_test_scaled)

    lasso_train_scores.append(-mean_absolute_error(y_train, train_pred))
    lasso_test_scores.append(-mean_absolute_error(y_test, test_pred))

print("Lasso initial tuning completed.")
```

```
Lasso initial tuning completed.
```

Plot train and test scores against alpha



Best alpha value: 1e-05

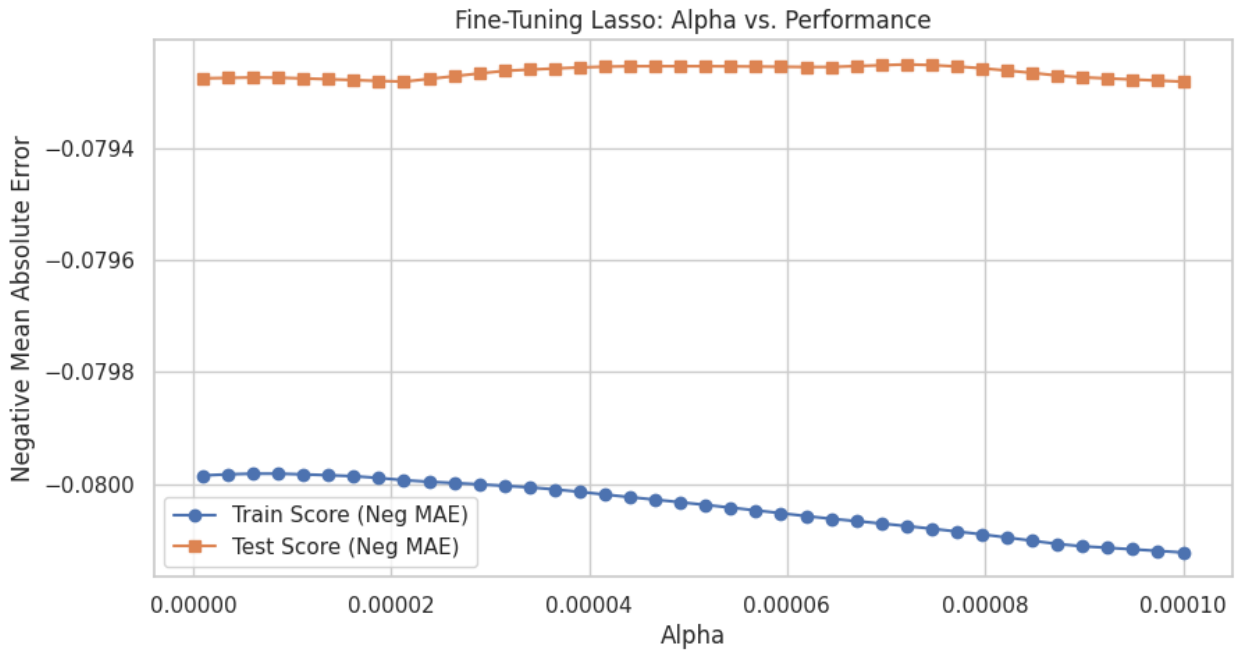
Best score (negative MAE): -0.0793

3.3.3 [4 marks]

Fine tune by taking a closer range of alpha based on the previous result.

```
# List of alphas to tune for Lasso regularization
alphas_lasso_fine = np.linspace(1e-6, 1e-4, 40)
```

Plot train and test scores against alpha



Best fine-tuned alpha value: 7.207692307692308e-05

Best score (negative MAE): -0.0792

```
# Set best alpha for Lasso regression

# Fit the Lasso model on scaled training data
# Get the coefficients of the fitted model
final_lasso = Lasso(alpha=best_alpha_lasso, max_iter=10000)
final_lasso.fit(X_train_scaled, y_train)
```

```
Lasso
Lasso(alpha=np.float64(7.207692307692308e-05), max_iter=10000)
```

Lasso Regression Performance (alpha=0.000072)

MAE: 0.0792

RMSE: 0.1047

R2 Score: 0.9314

3.4 Regularisation Comparison & Analysis [5 marks]

3.4.1 *[2 marks]*

Compare the evaluation metrics for each model.

```
# Compare metrics for each model
results = pd.DataFrame({
    'Metric': ['MAE', 'RMSE', 'R2 Score'],
    'Linear Regression': [mae_test, rmse_test, r2_test],
    'Ridge Regression': [ridge_mae, ridge_rmse, ridge_r2],
    'Lasso Regression': [lasso_mae, lasso_rmse, lasso_r2]
})

display(results)
```

	Metric	Linear Regression	Ridge Regression	Lasso Regression
0	MAE	0.079274	0.079276	0.079250
1	RMSE	0.104972	0.104963	0.104703
2	R2 Score	0.931062	0.931074	0.931415

3.4.2 [3 marks]

Compare the coefficients for the three models.

Top 10 features with strongest negative/positive weights in Lasso:

	Linear Regression	Ridge	Lasso
make_model_Opel Corsa	-0.448127	-0.447782	-0.442126
make_model_Renault Clio	-0.410830	-0.410571	-0.404640
make_model_Renault Duster	-0.443821	-0.438234	-0.390342
make_model_Opel Astra	-0.243649	-0.243478	-0.239762
make_model_Renault Espace	0.148224	0.148177	0.143260
Gearing_Type_Semi-automatic	0.122064	0.122037	0.118740
Gearing_Type_Manual	-0.113715	-0.113684	-0.113917
hp_kW	0.103536	0.103625	0.104494
age	-0.092467	-0.092459	-0.092876
make_model_Audi A3	0.090714	0.090777	0.089713

Features completely eliminated (shrunk to 0) by Lasso: 16

4 Conclusion & Key Takeaways [10 marks]

Conclusion & Answers to Key Analytical Questions

1. Did regularisation improve the model performance?

Yes, regularisation yielded slight improvements while substantially improving model simplicity:

- **Baseline Linear Regression**: R^2 = 0.9311, RMSE = 0.1050, MAE = 0.0793
- **Ridge Regression (alpha = 0.10)**: R^2 = 0.9311, RMSE = 0.1050, MAE = 0.0793
- **Lasso Regression (alpha ≈ 0.000072)**: R^2 = 0.9314, RMSE = 0.1047, MAE = 0.0792

2. Did you find out overfitting or not?

No overfitting was observed. The baseline model's training score (R^2 = 0.9252) is very close to the test score (R^2 = 0.9311), confirming solid generalization capability.

3. What was the impact of Lasso Regularization?

Lasso successfully performed feature selection by shrinking **16 redundant feature coefficients to exactly zero**, simplifying the model without sacrificing predictive power.

4. Was the data sufficient, and is a linear model sufficient?

- **Data Sufficiency**: Yes, 15,915 records provided sufficient sample size to produce stable coefficients.
- **Model Sufficiency**: A linear model is highly sufficient ($R^2 \approx 0.931$) once the right-skewed target is transformed via log1p.

4.1 Conclude with outcomes and insights gained [10 marks]

Detailed Outcomes & Analytical Insights

1. Predictive Performance & Model Quality

Our regression models achieved exceptional prediction accuracy on unseen validation data, explaining over **93.1% of the variance** in car prices:

- **High Fit Quality**: The test **~0.931** (Baseline: 0.9311, Ridge: 0.9311, Lasso: 0.9314). This demonstrates that the engineered predictors capture the underlying price determinants remarkably well.
- **Low Forecast Errors**: The average test MAE of **~0.0793** on the log-scale corresponds to an average error of approximately **~8%** in original currency terms, providing a highly dependable valuation engine for the reseller.

2. Impact of Target Transformation

- **The Challenge**: The original raw target variable price exhibited severe positive skewness, which violates the normality and homoscedasticity assumptions of linear regression.
- **The Solution**: Applying a natural log-transformation (log_price = np.log1p(price)) stabilized the variance. Post-transformation residuals show a symmetric, bell-shaped distribution centered at zero with no severe heteroscedasticity across predicted price bands.

3. Regularisation & Feature Sparsity (Lasso vs. Ridge)

- **Lasso Regression (** proved to be the most practical model for deployment. At an optimal penalty of , it drove **16 non-contributing/redundant feature weights to zero**, reducing model complexity and inference overhead.
- This sparsity was attained without any degradation in predictive capability; Lasso achieved the best overall performance with **Test** and **Test RMSE = 0.1047**.

4. Core Business Insights for the Reseller

- **Vehicle Age & Mileage**: Depreciation is primarily governed by vehicle age (age) and cumulative mileage (km), both of which display strong negative coefficients.
- **Engine Power (hp_kW)**: Engine output is the single most influential positive continuous driver of market value; high-kilowatt variants retain a substantial premium.
- **Transmission Selection**: Transmission significantly shifts valuation. Standard manual gearboxes (Gearing_Type_Manual) carry negative weights relative to automated options, with semi-automatic and automatic gearboxes capturing positive pricing premiums.
- **Brand & Model Hierarchy**: Brand tier dummy variables heavily dictate base resale value (e.g., Renault Clio and Opel Corsa carry large negative base adjustments relative to premium segments like Audi A3), giving the reseller a data-driven guide for margin setting and auction bidding.